

Ham Radio Accessibility Bridge

RX / Speech-to-Text Pipeline — Technical Overview

Branko Mihajlović · YU1FWX

Status as of early October 2026 · Active development

This document summarizes the architecture, engineering decisions, and validated findings behind the RX/STT (receive audio to live captions) half of the Ham Radio Accessibility Bridge project: a tool that lets Deaf and hard-of-hearing operators follow live ham radio conversations as readable, word-by-word text. It exists so that explaining the project to someone else doesn't depend on remembering every detail from scratch.

The short version: real HF radio audio is genuinely hard for off-the-shelf speech recognition — signals are often only 2-3dB above the noise floor. Most of the real engineering here is a custom, live-tested audio-cleaning pipeline built specifically for that problem, feeding two independent speech-to-text engines (one fully offline, one cloud-based) into a live, accessible transcript display.

1. Motivation & Scope

Amateur radio ('ham radio') nets are entirely audio-based, which locks out Deaf and hard-of-hearing operators by default. This project builds a bridge: live captions for receiving (RX), and a typed-text-to-speech path for transmitting (TX, not yet built). The RX/STT half was chosen as the starting milestone.

Strategic pivot: the project originally targeted a fully offline tool (no internet dependency, for emergency/off-grid use). Partway through, the scope was deliberately widened to also support an online mode (cloud speech recognition), reasoning that most real ham net traffic happens when internet access is available anyway, and a

more accurate tool sooner helps more people than holding out for a perfect offline version first. The offline goal was NOT abandoned — both modes now coexist, selected by a command-line flag.

2. System Architecture

The pipeline is a single Python script (`live_caption.py`) with two interchangeable speech-to-text engines sharing the same audio front-end:

- Local engine (default): fully offline, uses faster-whisper running on CPU, plus a parallel grammar-constrained recognizer (Vosk) specialized for ham jargon.
- Online engine (`--engine deepgram`): streams audio to Deepgram's cloud speech API. Currently the more accurate option on difficult (fast, accented) real speech, at the cost of requiring an internet connection and an API key.

Audio flows through five stages, in order, before reaching either engine:

- 1. Capture — from a physical radio line-in, or Windows audio loopback from the FlexRadio FlexRemote/SmartSDR app (used for all development/testing so far, zero cabling required).
- 2. Audio filter chain — the custom denoising pipeline (Section 4).
- 3. Resampling to 16kHz (the rate both speech engines expect).
- 4. Voice-activity segmentation — deciding where one utterance starts/ends.
- 5. Transcription — Whisper/Vosk locally, or Deepgram in the cloud — rendered into a live, word-by-word transcript window.

3. Audio Capture

Two capture modes share the same downstream pipeline, switchable with a command-line flag, no code changes needed:

- Loopback capture (current default) — captures whatever FlexRemote/SmartSDR is already sending to the PC's audio output. Easiest for development: no cable, works anywhere the radio software runs.
- Physical line-in — a real radio audio-out cable into a sound interface (e.g. an M-Audio Fast Track Pro, or a dedicated interface like a Signalink/DigiRig). Built and kept working from the start since it's the real long-term deployment path.

4. Audio Filter / Denoising Chain

Six independently-toggleable stages, always run in this fixed order. The order is deliberate — in particular, the Noise Gate must make its decisions from audio BEFORE Spectral Subtraction touches it, or Subtraction's own artifacts fool it (see the bug

note in Section 6).

Noise Blanker -> Bandpass -> Spectral Subtraction -> Noise Gate -> Spectral Gate -> DeepFilterNet3 (neural denoiser)

Validated live, against real HF radio recordings:

Spectral Subtraction

VALIDATED

Classical technique: tracks a per-bin noise-floor estimate over time and subtracts it from every frame. Known failure mode ('musical noise') mitigated with over-subtraction + a never-fully-null floor. Validated settings: over-subtract factor 2.0, floor 30%.

Noise Gate

VALIDATED

Custom-designed broadband gate: opens primarily on detected voice pitch (NOT on loudness alone — a plain level threshold was already proven unworkable on this signal, since a weak station can be fully intelligible while amplitude-indistinguishable from the noise floor). Threshold must stay very low (1-2dB) or it starts cutting real weak signal.

DeepFilterNet3

VALIDATED

A pre-trained neural speech-enhancement model (open source). Resets its own internal state every processing block (~0.5s), which can cause audible block-boundary clicks on certain interference — counterintuitively, keeping its suppression LIMIT low (~4dB) reduces this, rather than a higher limit giving a cleaner result.

Tried, and found NOT to help on real audio:

Bandpass Filter

NEGATIVE RESULT

Narrowing to the SSB voice band (300-3000Hz) sounded reasonable in theory, but measurably hurt transcription accuracy in real A/B testing, even on strong signal.

Noise Blanker

NEGATIVE RESULT

A hardware-style impulse (click/pop) suppressor, modeled on real radio noise blankers. Sounded too sharp/fast even after widening its settings for this net's interference.

Spectral Gate

NEGATIVE RESULT

A per-bin frequency gate, later rebuilt to be harmonic-aware (protecting bins that align with a detected voice pitch). Still no audible or measured transcription improvement after the rebuild.

An adaptive line-enhancement filter (NLMS) was also built and benchmarked, but A/B testing showed it made transcription dramatically worse (it fabricated words that were never spoken) — never wired into the live pipeline.

5. Speech-to-Text Engines

Local engine (offline, default)

- faster-whisper ('small', multilingual) — runs entirely on CPU. Hallucination

filtering is applied (Whisper, like most ASR models, can invent fluent-sounding text from pure noise) using confidence/compression-ratio heuristics.

- Voice-activity detection combines WebRTC's speech/noise classifier with the same pitch detector used in the audio filter chain — needed because fast, clipped phonetic-alphabet spelling (e.g. rapidly spelling a callsign) was found to silently fool the classifier alone into never triggering.
- Vosk (grammar-constrained parallel channel) — a second, lower-latency recognizer running alongside Whisper, restricted to a known ham-radio vocabulary (phonetic alphabet, Q-codes, procedural words). Inspired by how digital modes like FT8 gain robustness from a constrained message space.

Online engine (--engine deepgram)

- Streams audio to Deepgram's cloud API (Nova-2 model). Currently sounds noticeably better than the local engine on fast or accented real speech.
- Uses the same ham-radio vocabulary as a weighted keyword list, tuned per-word after live testing showed blanket boosting could backfire (e.g. hallucinating 'QRT' on pure noise).
- Requires an internet connection and an API key — this is the trade-off against the fully-offline local engine, not a replacement for it.

6. Key Engineering Findings

The recurring lessons that shaped how this project is built and tested:

- Real noise has no fixed pattern — it's random, not a shape you can learn once and subtract perfectly. Any denoising technique can therefore INVENT content that isn't there just as easily as it can reveal content that is. Confirmed concretely: Spectral Subtraction's own residual artifact ('musical noise') was measured to fool the Noise Gate's pitch detector into falsely registering it as real voice 17% of the time on pure noise, vs under 1% when analyzing the signal before Subtraction touched it. Fixed by having the Gate analyze the pre-Subtraction signal.
- A plain loudness/RMS threshold cannot reliably separate weak real signal from noise on HF radio — a marginal station can be fully intelligible to a human while amplitude-indistinguishable from the noise floor. Every detection mechanism in this project (VAD, the Noise Gate) is built around voice-pitch detection first, loudness only as a secondary signal.
- CPU oversubscription, not algorithm bugs, was the root cause of repeated 'freezing/garbled output' symptoms early on — three concurrent engines (Whisper, DeepFilterNet3, Vosk/Kaldi) competing for all CPU cores, with one library's thread pool completely uncapped. Each recurrence looked like a different bug until CPU usage was actually checked.
- A display/visualization bug can look identical to a real audio problem. A waterfall (spectrogram) display that auto-normalized every frame to its own

peak made every signal level look equally 'hot' regardless of actual strength — easy to misdiagnose as a real audio clipping issue until the actual transcription behavior was checked against it directly.

- Every new processing stage gets offline-tested against synthetic signals first, then against real recorded audio, then live — and several real bugs (hard-cut clicks, startup transients, a chunk-boundary gain-step click) were caught before ever reaching a live test this way. Trusting DSP theory or an isolated metric without a real-audio or real-transcript check has repeatedly been wrong in both directions: techniques that sounded right in theory turned out harmful (Bandpass, NLMS), and one technique with a confirmed real bug (DeepFilterNet3's state reset) still turned out net-positive anyway.

7. GUI & Live Monitoring Tools

- Word-by-word live transcript — text commits permanently as soon as a newer word appears after it, instead of waiting for a full phrase or silently rewriting itself. Built specifically because a Deaf/HoH user can't ask 'what did you just say?' the way a hearing person could — anything that flickers or gets overwritten before it's read is unrecoverable.
- Live waterfall (spectrogram) display — SDR-panadapter style, per-frequency-bin noise-floor-calibrated so weak signals stay visible regardless of overall band noise level, with a live pitch-tracking overlay.
- Audio monitor — lets the raw or filtered audio be played back and judged by ear, separate from the transcript, since filtering quality isn't always obvious from transcription output alone.
- Live SNR/RMS and approximate playback-latency readouts, plus on-screen sliders for every filter-chain parameter — tuning happens live, by ear and by number, without editing code and relaunching.

8. Standalone Audio Filter Library

Once the filter chain (Section 4) had real live-test evidence behind it, it was extracted into a separate, standalone Python package (`audio_filter.py`) with zero speech-recognition dependencies — no VAD, no ASR, no transcription code. It exposes the exact same six-stage chain as a plain, reusable class, so the denoising work from this project can be dropped into any other application that just wants cleaner audio out, including the author's separate FlexRadio-related tooling.

Ships with a parameter reference, the same validated-vs-negative findings table as Section 4, a runnable example script, and clear documentation of the one load-bearing design constraint (the Noise Gate's pre-Subtraction analysis order).

9. Current Status

Stage:	Active development and live-testing, not yet a finished product.
Validated chain:	Spectral Subtraction (over=2.0, floor=30%) + Noise Gate (threshold 1-2dB) + DeepFilterNet3 (atten limit ~4dB).
Off by default:	Bandpass filter, Noise Blanker, Spectral Gate — all confirmed not to help on this project's real test audio.
Engines:	faster-whisper + Vosk (offline, default) or Deepgram (online, --engine deepgram).
Known open items:	Whether Spectral Subtraction's residual artifact measurably confuses Whisper (not yet tested with real transcripts); a decision on capping/dropping stale transcript backlog under heavy band congestion; further live tuning of the Noise Gate's release time.

10. Technology Stack

Language:	Python
Speech recognition:	faster-whisper (local), Deepgram API (cloud), Vosk (grammar-constrained parallel channel)
Neural denoising:	DeepFilterNet3 (PyTorch)
DSP / audio:	NumPy, SciPy (FFT-based spectral processing, IIR filtering), WebRTC VAD, audioop (resampling)
Audio I/O:	sounddevice (line-in capture), soundcard (WASAPI loopback capture)
GUI:	Tkinter, Pillow (waterfall rendering)

Document generated to support explaining the current state of this project without needing to reconstruct it from memory each time.